

Real-Time Facial Expression Detection

A real-time facial expression (emotion) recognition system built with **Python**, **OpenCV**, and a **deep learning CNN** (**TensorFlow/Keras**). It detects faces from a live webcam feed and classifies each one into an emotion category with a confidence score, drawn live on the video.

Detected Emotions

Angry · Disgust · Fear · Happy · Sad · Surprise · Neutral

Project Structure

```
facial_expression_detection/  
├── data/                # Place fer2013.csv  
here (not included)  
├── model/              # Trained model saved  
here (emotion_model.h5)  
├── train_model.py      # Trains the CNN on  
FER-2013  
├── real_time_detection.py # Runs live webcam  
emotion detection  
├── requirements.txt    # Python dependencies  
└── README.md
```

How It Works

1. **Face detection** — OpenCV's Haar Cascade classifier locates faces in each webcam frame.

2. **Preprocessing** — each detected face is cropped, converted to grayscale, resized to 48×48 px, and normalized.
3. **Classification** — the preprocessed face is passed through a trained CNN that outputs a probability for each of the 7 emotion classes.
4. **Display** — a bounding box and the predicted emotion + confidence are drawn on the live video feed.

Setup (VS Code)

1. **Clone / copy the project folder** and open it in VS Code.
2. **Create a virtual environment** (recommended):

```
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # macOS / Linux
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

In VS Code, select this virtual environment as your Python interpreter
(`Ctrl+Shift+P` → *Python: Select Interpreter*).

4. **Get the training dataset** (only needed if you want to train your own model):
 - Download `fer2013.csv` from [Kaggle's FER-2013 dataset](#).
 - Place it in a `data/` folder in the project root.
5. **Train the model:**

```
python train_model.py
```

This saves the trained model to `model/emotion_model.h5`. Training on a CPU can take a while (a GPU is recommended); expect roughly 60–70% test accuracy, which is typical for FER-2013.

Already have a pretrained `.h5` model? Just drop it into `model/emotion_model.h5` and skip this step.

6. Run real-time detection:

```
python real_time_detection.py
```

A window will open showing your webcam feed with live emotion labels. Press `q` to quit.

Requirements

- Python 3.9–3.11
- A working webcam
- See `requirements.txt` for Python packages

Troubleshooting

Issue	Fix
Cannot open webcam	Check no other app is using the camera; try <code>cv2.VideoCapture(1)</code> if you have multiple cameras.
<code>emotion_model.h5</code> not found	Run <code>train_model.py</code> first, or supply a pretrained model.

Issue	Fix
Low FPS	Reduce frame resolution, or run inference every N-th frame instead of every frame.
Poor accuracy on your face	FER-2013 has known label noise/bias; consider fine-tuning on your own labeled data.

Possible Extensions

- Swap the Haar Cascade for a more accurate deep-learning face detector (e.g. MTCNN, MediaPipe).
- Use transfer learning (e.g. MobileNetV2) instead of a CNN trained from scratch.
- Log detected emotions over time for analytics (e.g. attention/engagement tracking).
- Deploy as a Flask/FastAPI web app streaming the annotated video to a browser.

License

Free to use and modify for educational and personal projects.